TTK4155
INDUSTRIAL AND EMBEDDED COMPUTER SYSTEMS DESIGN

Lab Lecture 3

02.10.2026

Tord Natlandsmyr

Lecture Overview

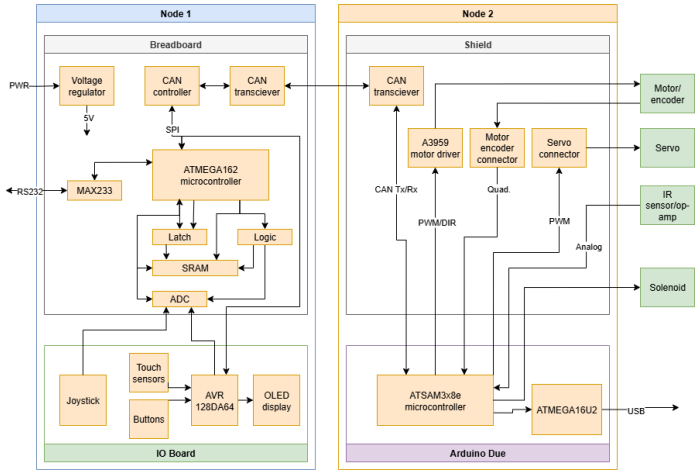
Recap 3
CAN Communication 6
Exercise 5: SPI & CAN Controller 13
Exercise 6: Introducing Node 2 25

Recap

Where are we?

Week	Exercise to be completed	Lab lecture
36	1: Initial assembly of microcontroller and RS-232	Lab Lecture 2, EL3
37	2: Address decoding and external RAM	
38	3: A/D conversion and joystick input	
39	4: OLED display and SPI part 1, IO buttons	
40	4: OLED display and SPI part 1, IO buttons	**Lab Lecture 3, EL3**
41	5: SPI part 2 and CAN controller	
42	6: Getting started with ARM cortex-M3 and communication between nodes	
43	Catch up week	Lab Lecture 4, EL3
44	7: Controlling servo and IR	
45	8: Controlling motor and solenoid	
46	Catch up week + Code delivery 13 Nov at 17:00	
47	Evaluation	

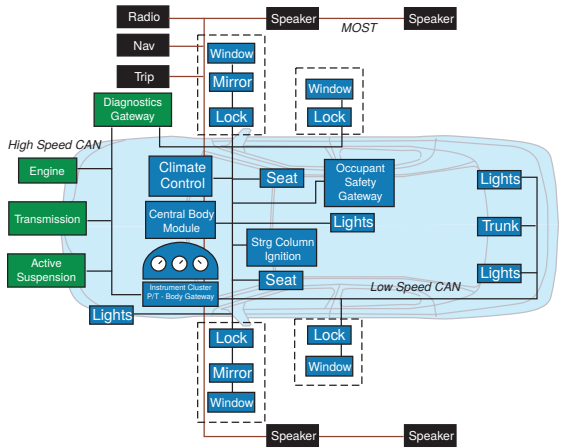
Node Communication



CAN Communication

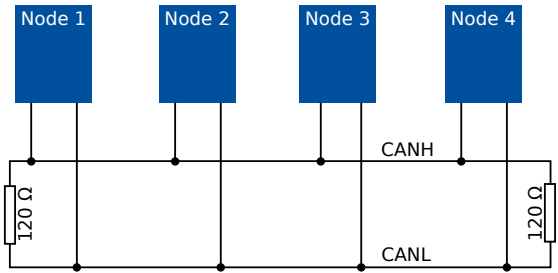
Controller Area Network (CAN)

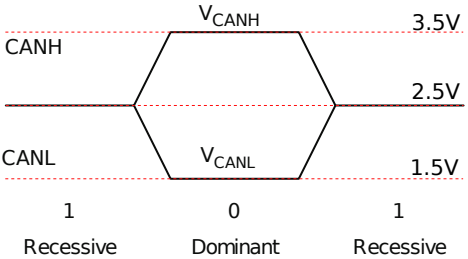
- Controller Area Network
- Vehicle bus = Noise resilient, delivery assurance etc.
- Multi-master broadcast bus protocol
- Message based 'addressing'
  - Nodes are easily added
- Arbitration without delay
- Limited datagram size
- Up to 1 Mbit/s
- For medium distances, < 40m
  - Longer for lower data rates
- Easy to use, but hard to set up



CAN Physical Layer

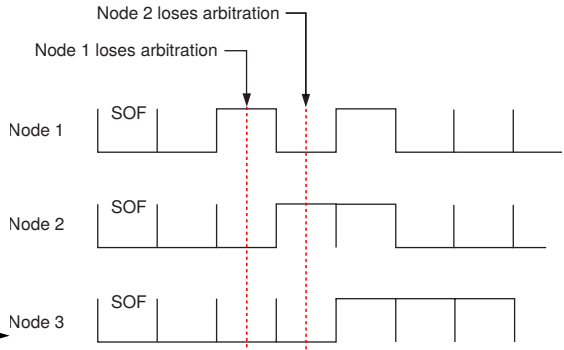
- Two wires, CANH and CANL



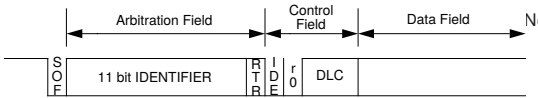


CAN Arbitration

- Frame: Identifier, IDE, DLC and Data fields are the most important
- Arbitration on ID
- IDs starting with the most 0 wins
- I.e. lowest/"most dominant" ID
- From CAN 2.0B spec

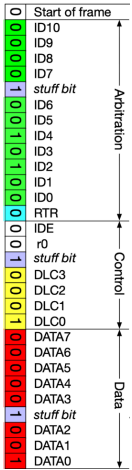


Standard Format



CAN Frame Breakdown

- SOF: Start of Frame
- ID: Identifier
- RTR: Remote Transmission Request
- IDE: ID exended bit
- DLC: Data Length Counter
- DATA: Data contents
- Hidden confusion: stuff bits after 5 similar bits
  - CRC, ACK, EOF are never stuffed
- Some unimportant fields after data



CAN Bit Timing

- Asynchronous $\Longrightarrow$ need other bit synchronisation method.
- Solution: Split bits into smaller segments defined by a number of 'quanta'.
- Rising edge acts as synchronisation segment.
- Signal is sampled between phase 1 and 2.
- MCP2515: Quantum length decided by tOSC and BRP
- Segment time should be the *same wall clock time* for each unit on the bus.
- **A lot of students get this wrong!**
- Read chapter 5 of the MCP2515 datasheet for more info



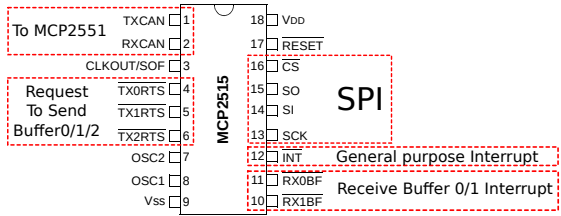
CAN Message ID

- Lowest ID: priority
- Broadcast $\Longrightarrow$ recipients filter irrelevant messages.
- Our responsibility: create a clever ID system.
- MASK: Determine which ID bits are (not) DC (Don't care).
- FILTER: Determine the accetable value of non-DC bits.
- See table 4-2 in MCP2515 for an example.

Exercise 5: SPI & CAN Controller

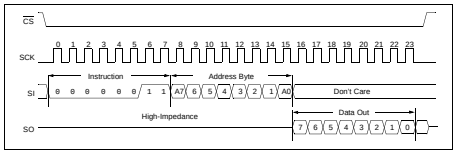
CAN Controller: MCP2515

- Uses SPI bus.
- Handles the framing stuff (transport layer).
- Controlled by writing and reading registers, much like the IO board units.
- External interrupts.
- Needs its own crystal.

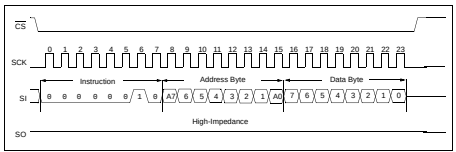


Using MCP2515

- Read



- Write



- SPI Interface
  - Select chip (CS= 0)
  - Send one byte instruction
  - Send/read additional bytes (address, bit mask, data)
  - Deselect chip(CS = 1)
- Section 12 of the datasheet contains instruction set.

MCP2515 Modes

- Configuration mode: Setup filters, masks and transceiver bit timings.
- Normal mode: Normal functionality.
- Sleep mode: Saves power when device is not used.
- Listen-only mode: Only receiving.
- Loopback mode: Internal transmission.
- Sections 10-11 detail modes and registers.
- CAN controller mode != SPI mode :)

CAN Configuration Summary

What information must be configured in the node on init, and what is encoded in each data frame?

Configuration

- Bit timing
- Mask and filter
- Mode

Sent via SPI on init

Frame

- ID, data length, etc
- The data itself

Sent via SPI for each CAN message

Tip: Make a suitable data structure

MCP2515 Transmission and Reception

Reception (Ch.4)

- Wait for a received message
  - Interrupt pin (enable CANINTE.RXnIE)
  - Read status registers (CANINTF.RXnIF)
- Read message
  - ID (RXBnSIDH & RXBnSIDL)
  - Data length (RXBnDLC)
  - Data (RXBnDM)
- Filter and Masks
  - RXBxCTRL.FILHIT(2:0) with RXFnSIDH, RXMnSIDH...

Transmission (Ch.3)

- Setup
  - Message ID (TXBnSIDH & TXBnSIDL)
  - Data length (TXBnDLC)
  - Data (TXBnDm)
- Request-to-send command. (TXBnCTRL.TXREQ)

Summary of Important MCP2515 Registers

- CANCTRL
- CNFn
- CANSTAT
- TXBnSIDH/TXBnSIDL
- RXBnSIDH/RXBnSIDL
- CANINTE/CANINTF
- TXBnDLC/RXBnDLC

Some Useful Functions

MCP/CAN controller

```
mcp2515_read();
mcp2515_write();
mcp2515_request_to_send();
mcp2515_bit_modify();
mcp2515_reset();
mcp2515_read_status();
```

SPI

```
SPI_send();
SPI_read();
SPI_init();
```

Remember to send dummy byte to read from slave MCP.

Header file for MCP2515 with register names and addresses is provided under 'Lab Tools' on the lab website

Example: Low Level

```
uint8_t mcp2515_read ( uint8_t address )
{
  uint8_t result;
  PORTB &= ~(1 << CAN_CS ); // Select CAN - controller
  SPI_write ( MCP_READ ); // Send read instruction
  SPI_write ( address ); // Send address
  result = SPI_read (); // Read result
  PORTB |= (1 << CAN_CS ); // Deselect CAN - controller
  return result;
}
```

Example: MCP Driver

```
uint8_t mcp2515_init () {
  uint8_t value;
  SPI_init (); // Initialize SPI
  mcp2515_reset(); // Send reset - command
  // Self-test
  mcp2515_read(MCP_CANSTAT, &value);
  if (( value & MODE_MASK ) != MODE_CONFIG ) {
    printf (" MCP2515 is NOT in configuration mode after reset !\n" );
    return 1;
  }
  // More initialization
  return 0;
}
```

Tips for Exercise 5

- CAN
  - Loopback mode: no messages visible on data lines
  - Chapter 3 of MCP2515 contains info on minimum setup
  - Hardest part of the project, probably
- To do steps
  1. Power-up CAN controller MCP2515(power+crystal...)
  2. Connect MCP2515 with Atmega162 via SPI
  3. Write MCP2515 driver
  4. Test CAN in loopback mode

Summary of CAN and Exercise 5

- MCP2515 datasheet chapters 2-5, 7, 10-12
- CAN 2.0B spec: Learning materials, strongly encouraged.
- CAN bus: asynchronous, arbitration on message ID.
- CAN nodes: must have identical bit timing settings.
- CAN frame, aka message: some 120 bits of metadata + data.
- CAN controller: MCP2515 communicates via SPI.
- E5: Get MCP2515 working in loopback mode, use the provided h-file.

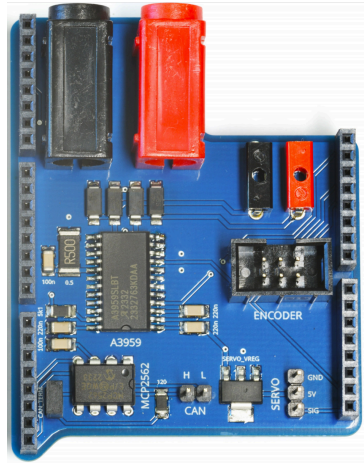
Exercise 6: Introducing Node 2

Introducing Node 2

- Arduino DUE with expansion card ("shield")
- We will **NOT** use the Arduino development framework
- ARM Cortex-M3 architecture: ATSAM 3X8E
  - 32-bit architecture (includes registers)
  - PMC: Power Management Controller. Most peripherals are deactivated by default. See table 9-1 in datasheet. You will need to activate the clock for the peripherals you will be using.
  - Integrated CAN controller
  - Beware of the watchdog (WDT)
- Has additional Atmega 16U2 that acts as an UART to USB bridge.
- ATSAM 3X8E programmed through JTAG.
- **3.3V logic (not 5v tolerant)**

Motor Shield

- Motor driver: Allegro A3959, Phase/Enable via Timer Counter PWM
- 3x2 encoder connector: quadrature signal via timer counter encoder
- MCP2562 CAN transceiver and terminator jumper
- 3x1 servo connector: angular position via timer counter PWM

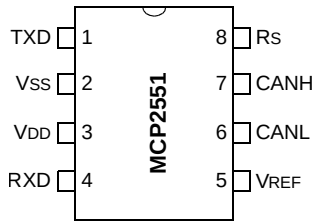


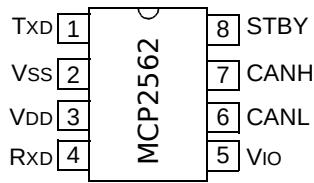
Node 2 Handout Code

- Basic UART and CAN drivers available on the lab website
  - Study and try to understand these before your lab day. Search the ATSAM3X8E datasheet for acronyms that you don't understand.
- CAN
  - INIT: You must find a fitting value for CAN_BR. Search in datasheet (40.9.6 and 40.7.4 w/Example at end). This must correspond to the value you set in the CNFx registers on MCP2515.
  - Interrupt handler example
- UART
  - Find/set the UART baud in uart.c

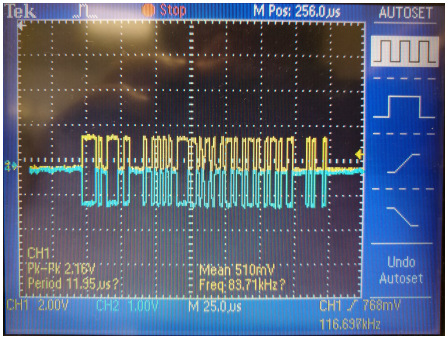
CAN Transceivers & Controllers

- CAN Controller MCP2515 and ATSAM3X8E integrated
- CAN Transceiver MCP2551 and MCP2562 (drop in replacement for MCP2551)
  - Handles the physical layer
  - Detects line errors
  - Protects against transients
  - End node termination of 120Ω
- Read AN228; A CAN Physical Layer Discussion
- Enable normal mode in Node 1





CAN Oscilloscope Example



Remember the stuffing bits!

CAN Driver: High Level

```
can_init()
can_message_send()
can_data_receive()
can_int_vect()
// Tip: Structs could be useful for CAN messages
struct can_message {
  uint16_t id ;
  char data_length ;
  char data [8];
};
```

Using Structs for CAN Messages

```
int main ( void ) {
  can_init();
  struct can_message message;
  message.id = 3;
  message.length = 1;
  message.data [0] = 'U ';
  can_message_send (& message );
}
void can_message_send ( struct can_message * msg ) {
  uint8_t i ;
  for ( i = 0; i < msg->length; i++)
}
```

Exercise 6 Tips

- Catch-up week!
- Be careful with CAN message transmission rate
  - Should be controlled/limited e.g. using a TIMER
- Hardware tips
  - Remember 120 ohm resistors at both ends
  - Use 22K ohm resistor for MCP2551 (Rs and GND)
- To do steps
  1. Connect CAN transceiver MCP2551 to node 1
  2. Program node 2 for CAN reception
  3. Verify transmission and reception by sending a dummy byte over CAN from node 1 to 2 and displaying it e.g. on UART on node 2
  4. Test CAN transmission between node 1 and 2. e.g. joystick position

Questions?

- E-mail: tord.natlandsmyr@ntnu.no
- Slides are available on Canvas

Thank you for your attention!

Next Lecture in Week 43